

Enterprise Portfolio Management System

Project Proposal for UCS503P

Submitted to: Nisha Thakur

Thapar Institute of Engineering and Technology

Simar Arora 1024030797

Bhanu Pratap 1024030787

Namit 1024030803

August 18, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the efficiency, transparency, and organization of portfolio management by providing a centralized platform for portfolio managers and clients. While the broader domain is investment and wealth management, the immediate engineering objective is to build a measurable, deployable software system that provides a consistent view of client portfolios, transactions, performance, and risk.

2 Time-to-value

... primary heuristic

- We will deliver the core portfolio workflow early by implementing authentication, client-manager assignment, portfolio creation, holdings, transactions, and basic valuation.
- Development will follow short iterations, with each iteration producing a functional increment that can be tested independently.
- CI-backed automated builds and tests will be introduced early to reduce integration and release friction.
- Later iterations will add market data, performance analytics, risk metrics, investment-request workflows, reporting, and AI-assisted explanations.
- Success will be measured by the correctness and responsiveness of the core portfolio workflow and improved through subsequent iterations.

3 Problem Statement

Portfolio managers handling multiple clients need to continuously monitor holdings, transactions, portfolio value, market prices, performance, allocation, and risk. Fragmented or manually managed workflows can create several problems:

- **Fragmentation:** client information, holdings, transactions, market information, and reports may be maintained across separate tools.
- **Limited visibility:** managers may not have a single, consistent view of the current state of each client's portfolio.

- **Manual calculations:** portfolio value, profit/loss, allocation, and performance can require repeated calculations.
- **Delayed information:** outdated prices can result in an inaccurate view of current portfolio value.
- **Inefficient workflow:** investment requests and manager decisions may require communication outside the portfolio-management system.
- **Limited risk visibility:** concentration, volatility, drawdown, and other indicators may not be available in one dashboard.
- **Reporting and audit overhead:** generating reports and tracing portfolio changes can require manual effort.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a web-based **Enterprise Portfolio Management System (EPMS)** with separate interfaces for portfolio managers and clients. The system will provide:

- **Client portal:** view portfolio holdings, value, performance, transactions, reports, and submit investment requests.
- **Manager portal:** monitor assigned clients, review portfolios, analyze performance and risk, and approve or reject investment requests.
- **Portfolio engine:** maintain holdings, transactions, cash balances, portfolio value, cost basis, and profit/loss.
- **Market-data layer:** obtain current market prices through a supported provider or simulated data during development.
- **Analytics and reporting:** calculate portfolio performance, allocation, selected risk metrics, and generate reports.
- **Audit and notifications:** record important portfolio actions and notify users of relevant workflow events.

4.2 Core workflow

1. Client logs in and views the current portfolio.
2. System displays holdings, portfolio value, P&L, allocation, and performance.
3. Client submits an investment request for a selected asset and amount.
4. Manager reviews the request together with portfolio and risk information.
5. Manager approves or rejects the request.
6. An approved request creates a simulated transaction and updates the portfolio.
7. Client receives a notification and can view the updated portfolio.

4.3 Operational constraints for an educational portfolio-management setting

- The platform will not execute real-money trades; transactions will be simulated.
- Market-data availability will depend on the selected provider, with mock data available for testing.
- Authoritative financial calculations will be performed by the backend rather than the AI model.
- AI will be used for explanations of verified application data, not autonomous trading decisions.
- The implementation will prioritize workflow correctness, security, deployability, and measurable performance over research-level financial algorithms.

5 Solution Approach

... Engineering focus

This is an engineering course project, so the focus is on reliable portfolio-management workflows, correctness, security, scalability, and measurable delivery rather than novel financial-algorithm research.

5.1 Portfolio calculation and analytics

... non-research

The backend will maintain clients, portfolios, assets, holdings, transactions, and cash balances. It will calculate portfolio value, cost basis, realized and unrealized P&L, return, and asset allocation. Later iterations will add volatility, Sharpe ratio, beta, maximum drawdown, concentration analysis, and benchmark comparison.

5.2 Web architecture

... web-based solution

A layered web architecture will be used:

- **Frontend:** role-specific interfaces for clients and portfolio managers.
- **Backend API:** authentication, authorization, portfolio management, transactions, requests, calculations, analytics, and notifications.
- **Database:** persistent storage for users, portfolios, holdings, transactions, and audit information.
- **Supporting services:** market-data access, caching where appropriate, and AI-assisted explanations.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to automatically build the application, run unit and integration tests, validate pull requests, and produce repeatable deployment artifacts. Development will use Git branches and pull requests rather than direct changes to the stable main branch.

6 Evaluation Criterion

... measurable and attributable

We define success using measurable metrics tied to the core portfolio workflow.

6.1 Primary evaluation metric

Portfolio valuation correctness: for predefined portfolios containing known holdings, transactions, and prices, calculated portfolio value and P&L should match independently calculated expected values within a predefined tolerance. Results will be attributable to stored transactions, quantities, cost basis, and market prices.

6.2 Secondary evaluation metrics

- **API response time:** response time for common operations such as login and portfolio retrieval.
- **Market-data latency:** time between receiving a price update and displaying the corresponding value.
- **Workflow correctness:** investment requests follow the expected PENDING → APPROVED/REJECTED transitions and approved requests update simulated records.
- **Access-control correctness:** clients can access only their own portfolios and managers only their assigned portfolios.
- **Reliability:** successful execution of the core workflow during the pilot/staging period.

6.3 Pilot validation plan

... high-level

- Use 2–3 simulated portfolio managers and 5–10 simulated clients.
- Create multiple portfolios, assets, transactions, and predefined market-price scenarios.
- Test login, portfolio viewing, investment requests, approval/rejection, portfolio updates, P&L recalculation, and access-control rules.

7 Scalability

... with foresight

The system should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** support growth in clients, portfolios, transactions, and concurrent dashboard usage through separation of frontend/backend, stateless APIs where appropriate, database indexing, pagination, and caching.
2. **Operationally deployable (practically):** containerize the application using Docker and use CI/CD for repeatable builds and deployments on commonly available hosting infrastructure.

8 Engine availability heuristic

A mature set of tools and services is available to implement the project as a web-based system:

- Web framework: React + TypeScript for the frontend and Java + Spring Boot for backend APIs.
- Relational database: PostgreSQL with Hibernate / Spring Data JPA.
- Authentication: Spring Security + JWT.
- Cache: Redis where appropriate.

- Market data: a supported provider such as DhanHQ, subject to access requirements.
- AI: Ollama with a locally hosted language model for explanations.
- Testing: JUnit and integration testing.
- CI/CD and deployment: GitHub Actions and Docker.

We will use these mature components to focus on workflow design, correctness, security, measurement, and deployability rather than inventing new infrastructure.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Role-based authentication and client/manager profiles.
- Client-manager assignment and portfolio creation.
- Asset, holding, and transaction management.
- Basic portfolio valuation and P&L calculation.
- Initial manager dashboard and client portfolio view.
- Backend unit tests and CI build/test pipeline.

9.2 Subsequent deliverables

- Market-data integration, current-price valuation, performance charts, and allocation.
- Investment requests, manager approval/rejection, simulated transactions, notifications, and audit logging.
- Risk metrics including volatility, Sharpe ratio, beta, maximum drawdown, and concentration analysis, together with portfolio reports.
- AI-assisted portfolio explanations, Dockerized deployment, performance/security testing, and CI/CD deployment.

10 Risks and mitigations

- **Market-data dependency:** use an abstraction layer and mock market data for development and testing.
- **Financial calculation errors:** use deterministic backend calculations and validate them with predefined test cases and independent expected values.
- **AI hallucination:** keep financial calculations in the backend and use AI only for explanations based on verified data.
- **Unauthorized access:** implement role-based authorization, ownership checks, protected APIs, and security tests.
- **Team integration conflicts:** use feature branches, pull requests, code review, automated tests, and a protected main branch.
- **Evaluation ambiguity:** define measurable metrics and reproducible test scenarios before development begins.

11 Summary

This project proposes a deployable web platform that centralizes portfolio management, market monitoring, performance analysis, risk analysis, client-manager workflows, reporting, and AI-assisted explanations. It follows the course's engineering focus by delivering a functional core workflow early, using CI/CD for reliable iteration, and evaluating the system through measurable correctness, responsiveness, security, and reliability criteria. The platform will provide two primary user experiences: a portfolio manager portal for managing clients and portfolios and a client portal for monitoring investments and interacting with the portfolio-management workflow.